



UNIVERSITÉ DE CORSE

ANNÉE UNIVERSITAIRE 2024-2025

Rapport de Projet

PYTHQUEST

Présenté par :
Léandre RAETH, Thomas SANNA

Projet présenté le 05/04/2025

L3 SCIENCES ET TECHNOLOGIES, PARCOURS INFORMATIQUE

Table des matières

1	Introduction	2
1.1	Présentation du projet	2
1.2	Déroulement du jeu	2
2	Conception orientée objet	3
2.1	Classes Principales	3
2.2	Cas d'utilisation principaux	4
3	Technologies utilisées	5
3.1	Nos bonnes pratiques	5
4	Structuration du code : MVC	6
5	Base de données	7
6	Interface graphique	8
7	Suivi de projet	10
7.1	Gestion de projet	10
7.2	Lien du dépôt Git	10
7.3	Versions du projet	10
7.4	Corrections mineures, corrections de bugs	12
7.5	Rééquilibrage du jeu	12

1 Introduction

1.1 Présentation du projet

PythQuest est un jeu de rôle textuel développé en Python, conçu dans le cadre du projet de fin de semestre. Ce jeu met en œuvre une architecture MVC (Modèle-Vue-Contrôleur) pour séparer la logique métier, l'interface utilisateur et la gestion des interactions.

Le joueur incarne un aventurier qui doit accomplir des quêtes, explorer des donjons, combattre des monstres et gérer son inventaire pour progresser dans le jeu. Les principales fonctionnalités incluent :

- **Gestion des quêtes** : Le joueur peut accepter des quêtes, les accomplir en battant des monstres spécifiques, et recevoir des récompenses en or et en expérience.
- **Exploration de donjons** : Les donjons contiennent des monstres aléatoires que le joueur peut combattre pour gagner des ressources.
- **Système de combat** : Le joueur peut attaquer les monstres, utiliser des potions pour se soigner, ou fuir les combats.
- **Gestion de l'inventaire** : Le joueur peut consulter ses armes, potions et équipements, et choisir une arme à équiper.
- **Interactions avec des PNJ** : Le joueur peut acheter des armes auprès d'un forgeron ou des potions auprès d'un médecin.

1.2 Déroulement du jeu

Le joueur crée son compte, ou se connecte avec son email et son mot de passe. Son personnage est alors créé à la suite de l'entrée de son nom.

Le joueur commence niveau 1. Son objectif est de monter de niveau en accomplissant des quêtes créées aléatoirement par le jeu.

Les quêtes consistent à battre un monstre spécifique qui se trouve dans un donjon regroupant plusieurs autres monstres aléatoires. Le joueur peut choisir de combattre ces monstres pour gagner de l'or et de l'expérience, ou de fuir.

Lors de l'entrée d'un donjon, des monstres apparaissent aléatoirement à tour de rôle. Le joueur peut choisir d'attaquer, de fuir ou d'utiliser une potion pour se soigner. S'il choisit d'attaquer, il peut infliger des dégâts au monstre et recevoir des coups en retour, selon l'arme qu'équipe le monstre face à lui.

Lorsqu'un monstre est vaincu, le joueur gagne de l'expérience, de l'or, et parfois une arme que le monstre possédait. Si le monstre vaincu est celui requis pour une quête en cours, la quête est marquée comme terminée, et le joueur reçoit une récompense supplémentaire en or et en expérience.

Si le joueur subit trop de dégâts et que ses points de vie tombent à zéro, il est renvoyé au village. Dans ce cas, il perd une partie de son or et doit se soigner ou acheter des potions auprès du médecin pour continuer son aventure.

Le joueur peut également interagir avec des PNJ (personnages non-joueurs) dans le village :

- **Le forgeron** : Permet d'acheter ou de vendre des armes, et parfois de forger de nouvelles armes.
- **Le médecin** : Vend des potions de soin pour restaurer les points de vie du joueur.

Le jeu propose une progression basée sur le niveau du joueur. À chaque montée de niveau, le joueur devient plus puissant, ce qui lui permet d'affronter des donjons et des monstres de plus grande difficulté. Les quêtes et les récompenses s'adaptent également au niveau du joueur, offrant un défi constant.

Il n'y a pas d'objectif final à proprement parler. Le joueur gagne du niveau à l'infini, et il y aura toujours des quêtes à faire au fil de l'aventure.

2 Conception orientée objet

2.1 Classes Principales

Le projet PythQuest repose sur une conception orientée objet, avec une organisation en plusieurs classes principales pour modéliser les différents éléments du jeu. Voici un aperçu des classes principales :

- **Personnage** : Classe de base représentant un personnage dans le jeu. Elle contient des attributs communs tels que le nom, les points de vie et l'or. Les classes spécifiques comme *Combattant*, *Monstre*, *Forgeron* et *Médecin* héritent de cette classe.
- **Combattant** : Représente le joueur. Cette classe gère les actions du joueur, telles que l'attaque, l'utilisation de potions, la gestion de l'inventaire, et l'accomplissement des quêtes. Elle contient également des attributs comme le niveau, l'expérience, et l'arme équipée.
- **Monstre** : Représente les ennemis que le joueur peut combattre dans les donjons. Chaque monstre possède un niveau, des points de vie, une arme, et une récompense en or.
- **Quête** : Modélise les quêtes que le joueur peut accepter. Une quête est associée à un monstre cible, un donjon, une difficulté, et une récompense en or. Elle contient également un statut indiquant si la quête est en cours ou terminée.
- **Donjon** : Représente les donjons explorables par le joueur. Chaque donjon contient une liste de monstres et est associé à un niveau de difficulté.
- **Arme** : Représente les armes que le joueur ou les monstres peuvent utiliser. Chaque arme a un nom, des dégâts, et une valeur en or.
- **Forgeron** : PNJ permettant au joueur d'acheter, de vendre ou de forger des armes. Il gère également un stock d'armes disponibles.
- **Médecin** : PNJ permettant au joueur d'acheter des potions pour restaurer ses points de vie. Il gère un stock limité de potions.
- **GestionnaireDeQuêtes** : Classe utilitaire responsable de la création et de la gestion des quêtes. Elle génère des quêtes aléatoires adaptées au niveau du joueur.

Chaque classe est conçue pour être modulaire et réutilisable, avec des méthodes

spécifiques pour gérer les interactions entre les différents éléments du jeu, et leur création.

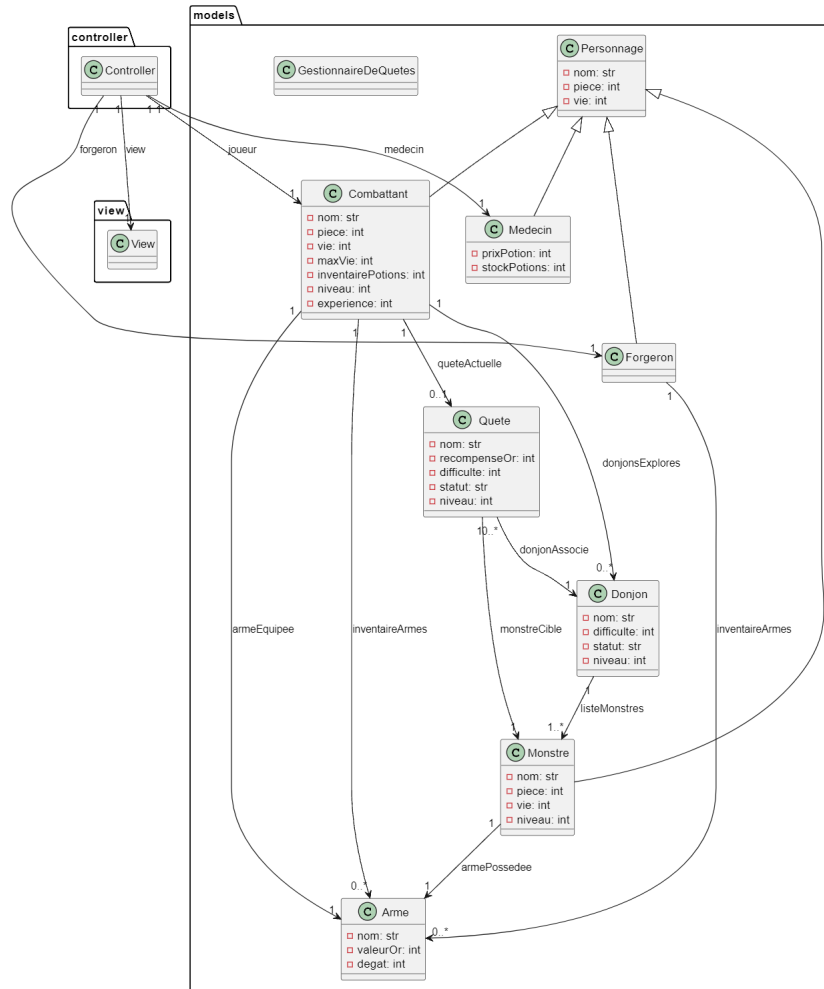


FIGURE 1 – Diagramme UML du projet PythQuest sans DAO

2.2 Cas d'utilisation principaux

Voici les principaux cas d'utilisation du jeu PythQuest :

- **Créer un personnage** : Le joueur peut créer un nouveau personnage en entrant son nom, son email et son mot de passe. Une fois créé, le personnage commence au niveau 1 avec un équipement de base.
- **Accepter une quête** : Le joueur peut consulter les quêtes disponibles

et en accepter une. Chaque quête est associée à un monstre spécifique à vaincre et offre des récompenses en or et en expérience.

- **Explorer un donjon** : Le joueur peut entrer dans un donjon pour affronter des monstres aléatoires. Chaque donjon est associé à un niveau de difficulté et contient des monstres adaptés au niveau du joueur.
- **Combattre un monstre** : Lors d'une exploration de donjon, le joueur peut engager un combat avec un monstre. Il peut choisir d'attaquer, d'utiliser une potion pour se soigner ou de fuir.
- **Acheter des armes** : Le joueur peut interagir avec le forgeron pour acheter de nouvelles armes, vendre des armes inutilisées ou forger des armes plus puissantes.
- **Acheter des potions** : Le joueur peut interagir avec le médecin pour acheter des potions de soin afin de restaurer ses points de vie.
- **Gérer l'inventaire** : Le joueur peut consulter son inventaire pour voir les armes et potions qu'il possède. Il peut également équiper une arme pour l'utiliser en combat.
- **Sauvegarder et charger une partie** : Le joueur peut sauvegarder sa progression lors de sa déconnexion et la charger ultérieurement pour continuer son aventure à sa connexion.
- **Abandonner une quête** : Si le joueur ne souhaite plus poursuivre une quête en cours, il peut l'abandonner pour en choisir une autre.

3 Technologies utilisées

Le projet a été développé en Python (d'où le nom PythQuest), en utilisant la bibliothèque Tkinter pour l'interface graphique, et mysql pour la gestion de la base de données.

Le code est organisé selon le modèle MVC (Modèle-Vue-Contrôleur) pour une meilleure séparation des préoccupations. Le modèle gère la logique métier et les données, la vue s'occupe de l'interface utilisateur, et le contrôleur gère les interactions entre le modèle et la vue.

Le code est structuré et vérifié à ce que les classes soient bien séparées et que les méthodes soient cohérentes. Des commentaires sont présents pour expliquer le fonctionnement de certaines parties du code.

3.1 Nos bonnes pratiques

Ce projet suit plusieurs bonnes pratiques de codage pour garantir la maintenabilité, la lisibilité et la robustesse du code. Voici les principales pratiques adoptées :

- **Nommage des variables et des fonctions** :
 - Utilisation de noms explicites et significatifs pour les variables et les fonctions.
 - Adoption du style *camelCase* pour les noms de variables et de fonctions (ex : `nomDeVariable`, `nomDeFonction`).

- Utilisation du style *PascalCase* pour les noms de classes (ex : `NomDeClasse`).
- Utilisation du style *snake_case* pour les noms de constantes (ex : `NOM_DE.CONSTANTE`).
- Éviter les noms de variables génériques comme `x`, `y`, `temp`, etc.
- **Structure du code :**
 - Séparation de la logique applicative et de l'interface utilisateur grâce au modèle MVC (Modèle-Vue-Contrôleur).
 - Organisation du code en modules et fichiers logiques.
 - Documentation des classes, méthodes et fonctions à l'aide de *docstrings*.
 - Utilisation de constantes pour éviter les valeurs en dur dans le code.
- **Structure MVC :**
 - Le modèle (*Model*) contient la logique métier et les données.
 - La vue (*View*) gère l'affichage et l'interface utilisateur.
 - Le contrôleur (*Controller*) gère les interactions entre le modèle et la vue.
 - Éviter de mélanger la logique métier avec l'interface utilisateur.
 - Regrouper tout ce qui concerne une classe dans le même fichier (ex : `Combattant` dans `models/combattant.py`).
 - Organisation des fichiers : les classes de modèles dans `models/`, les classes de vues dans `view/`, et les classes de contrôleurs dans `controller/`.
- **Gestion des exceptions :**
 - Gestion appropriée des exceptions pour éviter les plantages inattendus.
 - Création d'exceptions personnalisées pour les erreurs spécifiques à l'application (dans `models/exceptions.py`).
 - Utilisation de blocs `try-except` pour capturer et gérer les erreurs spécifiques.
- **Tests automatisés :**
 - Écriture de tests unitaires pour vérifier le bon fonctionnement des différentes parties du code.
 - Utilisation de frameworks de test comme `unittest` ou `pytest` pour automatiser les tests.
- **Documentation :**
 - Documentation du code avec des commentaires et des *docstrings*.
 - Utilisation d'annotations de type pour spécifier les types de paramètres et de retour des fonctions.
 - Maintien d'un fichier `README` à jour avec des instructions claires pour l'installation et l'utilisation du projet.

4 Structuration du code : MVC

Comme expliqué précédemment, le projet est structuré selon le modèle MVC (Modèle-Vue-Contrôleur). Voici un aperçu de la structure du code :

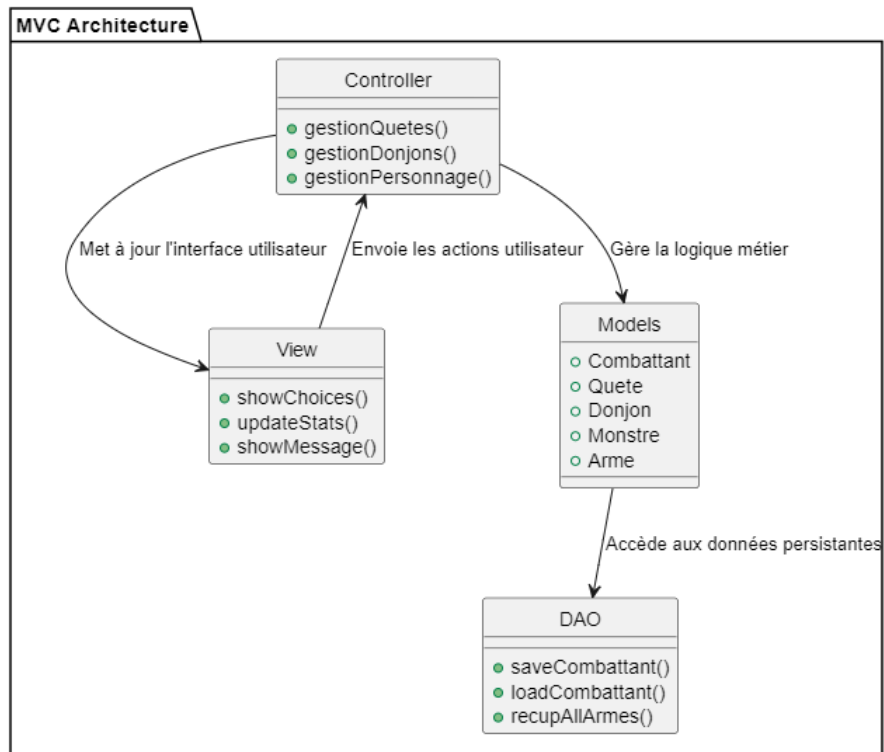


FIGURE 2 – Structure du code selon le modèle MVC

5 Base de données

PythQuest est un jeu pensé hors-ligne. Mais nous avons tout de même implémenté une base de données sur un serveur et non en local, pour nous permettre une compréhension de la gestion des bases de données en Python.

En conséquence que le jeu soit hors-ligne, aucune donnée d'un joueur n'interfère avec un autre joueur. Chaque joueur possède ses propres armes, son forgeron, etc...

Voici donc le "Concepteur" de base de données :

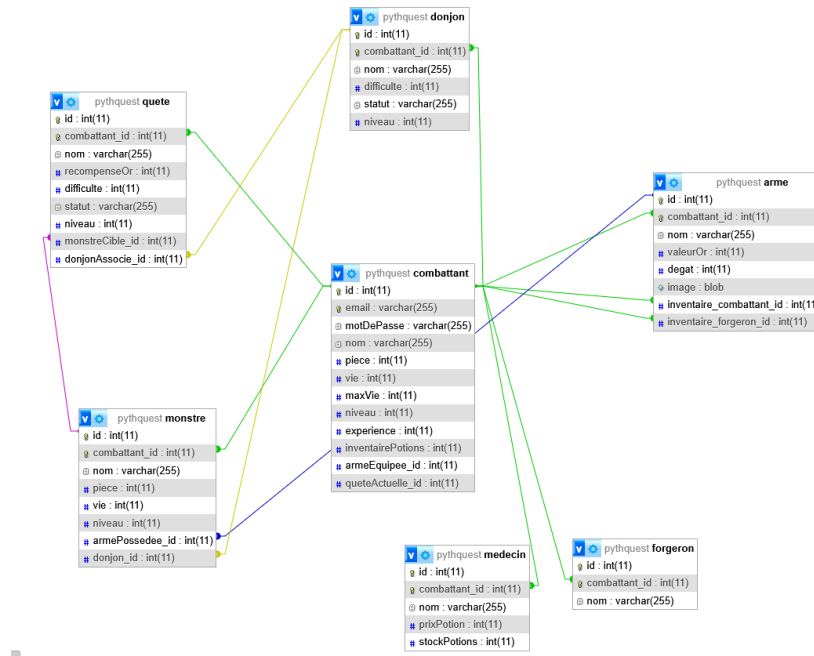


FIGURE 3 – Concepteur de la base de données

En effet, on peut voir que l'unicité d'un élément d'une table, par exemple "arme", est gérée par l'ID de l'arme, mais aussi par l'id du joueur.

Cela permet de s'assurer qu'aucun joueur n'aura la même arme qu'un autre joueur.

6 Interface graphique

L'interface graphique du jeu a été développée à l'aide de la bibliothèque Tkinter. Elle permet d'interagir avec le joueur de manière intuitive et agréable. Voici un aperçu de la fenêtre principale du jeu :

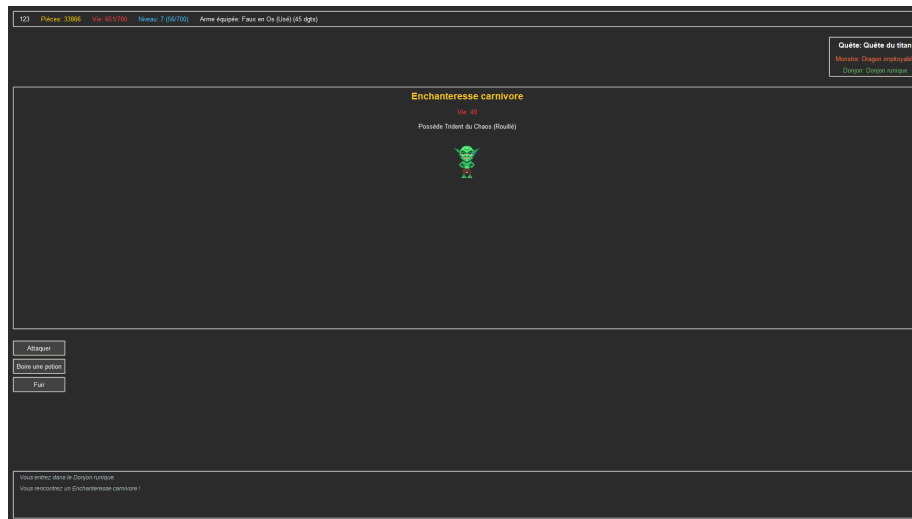


FIGURE 4 – Fenêtre principale du jeu

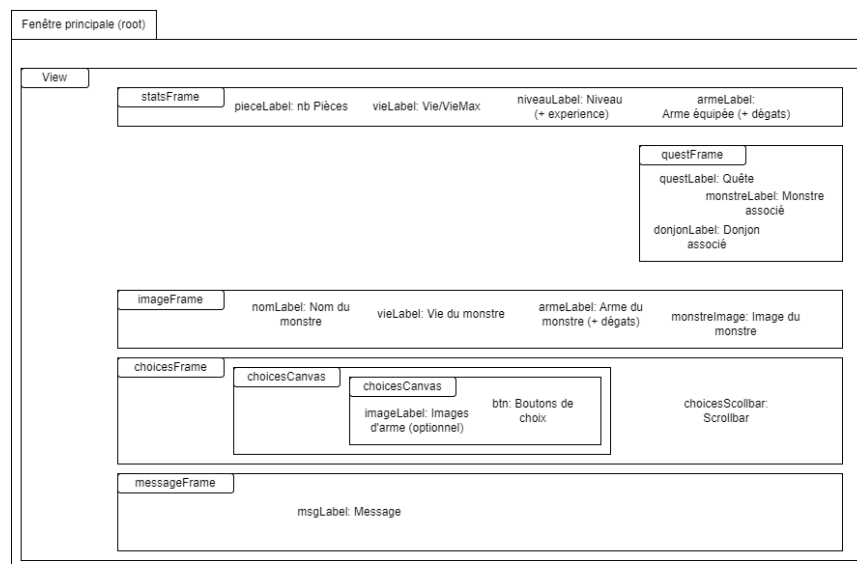


FIGURE 5 – Structure de l'interface du jeu

7 Suivi de projet

7.1 Gestion de projet

Le projet a été géré de manière collaborative, avec une répartition des tâches entre les membres de l'équipe. Nous avons utilisé un système de gestion de version (Git) pour suivre les modifications du code et faciliter la collaboration. Nous avons également utilisé Discord pour prévenir de certains changements et les expliquer.

7.2 Lien du dépôt Git

Le code source du projet est disponible sur GitHub. Vous pouvez y accéder en suivant ce lien : <https://github.com/ThomasSanna/projet-POO-L3STI>

Voici les différentes branches du projet :

- **V1** : Version 1.0 du projet
- **V2** : Version 2.0 du projet
- **V3** : Version 3.0 du projet
- **VFinale** : Version finale du projet
- **Main** : Version principale du projet, qui est la version 4.0 du projet. Cette branche n'a contenu que des versions fonctionnelles. Si une version ne fonctionnait pas, elle était mise dans une autre branche.
- **Thomas** : Branche de Thomas, qui a été utilisée pour le développement de certaines fonctionnalités du jeu.
- **Leandre** : Branche de Léandre, qui a été utilisée pour le développement de certaines fonctionnalités du jeu, et pour la gestion de l'UI
- **UIThomas** : Branche de Thomas, qui a été utilisée pour le développement de l'UI du jeu.
- **ThomasConn** : Branche de Thomas, qui a été utilisée pour le développement de la connexion du jeu et des sauvegardes.

7.3 Versions du projet

Le projet a été développé en 4 versions principales, chacune correspondant à une étape clé de son développement. Voici un aperçu des principales versions et des fonctionnalités ajoutées à chaque étape :

- **Version 1.0** : Création de la structure de base du projet, avec les classes principales (Personnage, Combattant, Monstre, Quête, Donjon, Arme). Mise en place des premières fonctionnalités de jeu sous forme vue console.

Toute cette version a été fait en Pair Programming. Léandre a souvent pris le rôle du "Navigateur" (celui qui propose des idées et des solutions), tandis que Thomas a pris le rôle du "Conducteur" (celui qui tape le code). Nous avons alterné ces rôles au fil du projet, mais nous avons souvent gardé cette répartition.

- Gestion de quêtes (acceptation, accomplissement, abandon)
- Exploration de donjons

- Combat contre des monstres
- Gestion de l'expérience et des niveaux
- Gestion de l'inventaire et de l'équipement d'arme
- Gestion de l'or et des récompenses lors d'une fin de quête ou d'une victoire contre un monstre
- Gestion de la mort du joueur et pénalisation de l'or
- **Version 2.0 :**
 - Nous nous sommes attribués des tâches à faire chacun de note côté. Pour chaque cas ci-dessous, les informations sur qui a fait quoi sont indiquées, avec informations supplémentaires.*
 - Ajout de commentaires et de docstrings pour améliorer la lisibilité du code. [Thomas]
 - Mise en place du système MVC pour séparer la logique métier et l'interface utilisateur à l'aide d'un contrôleur. [Thomas]
 - Ajout de l'interface graphique avec Tkinter pour améliorer l'expérience utilisateur. [Léandre. Avec l'aide de Thomas pour lier au contrôleur]
 - Création aléatoire des quêtes et des monstres dans les donjons. [Thomas]
 - Ajout de la gestion des niveaux de difficulté des quêtes et des monstres. [Thomas]
 - Implémentation du forgeron et achat d'arme [Thomas]
 - Implémentation du médecin et achat de potions [Thomas]
 - Ajout d'images personnalisées pour les armes. [Léandre]
- **Version 3.0 :**
 - Nous avons continué à nous répartir les tâches.*
 - Ajout de tests unitaires pour valider le bon fonctionnement de la classe Combattant et de certaines de ses méthodes. [Thomas]
 - gagnerExperience
 - resetApresMort
 - gagnerPotion
 - perdrePotion
 - boirePotion
 - acheterPotion
 - acheterArme
 - accepterQuete
 - abandonnerQuete
 - battreMonstre
 - attaquer
 - Implémentation de la base de données [Thomas]
 - Ajout de la gestion de la connexion et de la création de compte (Combattant et son inventaire d'armes et potions) [Thomas]
 - Ajout de la gestion de la sauvegarde et du chargement de la partie (Combattant et son inventaire d'armes et potions) [Thomas]
- **Version Finale :** Optimisation du code et ajout de commentaires pour améliorer la lisibilité. Tests finaux et corrections de bugs. Préparation du rapport final.

7.4 Corrections mineures, corrections de bugs

Le projet a été testé tout au long de son développement, et plusieurs bugs ont été corrigés. Voici quelques exemples de bugs identifiés et corrigés :

- **Version 1.0 → Version 2.0 :**
 - Correction du fait que le joueur pouvait battre un monstre qui appartenait à une quête non-associée, sans que la quête ne soit notée comme "Terminée".
 - Optimisation de la création de donjon :
 - Empêcher la création d'un donjon qui possède la même difficulté qu'un donjon déjà en cours.
 - Correction du fait que le combat contre un monstre d'une quête associée ne se terminait pas correctement.
 - Affichage de la récompense en or de la quête, ainsi que l'affichage de la récompense d'arme et d'or lorsqu'on bat un monstre.
 - Correction du fait que le joueur n'arrivait pas à fuir un donjon.
 - Correction du fait que le joueur pouvait avoir une valeur négative d'expérience lors d'un passage de niveau.
 - Correction de certaines méthodes dans Forgeron et Combattant qui utilisaient encore des `print()` pour afficher des informations.
- **Version 2.0 → Version 3.0 :**
 - Correction du fait que les données sur l'écran ne se mettaient pas à jour lors d'un certain cas à l'achat de potion et de vie.
 - Ajout du nom du joueur dans l'interface graphique.
 - Ajout de plusieurs icônes pour représenter les monstres lors d'un combat.
- **Version 3.0 → Version Finale :**
 -

7.5 Rééquilibrage du jeu

Pour qu'un jeu soit appréciable, il faut impérativement qu'il soit équilibré.

- **Version 1.0 → Version 2.0 :**
 - Baisse de la récompense d'or des monstres de 20%.
 - Baisse des dégâts des monstres (3 à 10 → 2 à 5 (évoluant avec la difficulté et le niveau du joueur)).
 - Baisse de la vie maximale obtenue lors d'un passage de niveau, qui était beaucoup trop élevée au fil du temps ($VieMax \cdot 1.5 \rightarrow VieMax + 100$ à chaque niveau).
 - Augmentation du prix des armes chez le forgeron de 50%.
- **Version 2.0 → Version 3.0 :**
 - Baisse de la scalabilité des dégâts des monstres selon le niveau du joueur ($Dégâts \cdot niveau \rightarrow Dégâts \cdot \sqrt{niveau}$).
 - Baisse de la scalabilité des dégâts des monstres selon la difficulté du donjon ($Dégâts \cdot difficulté \rightarrow Dégâts \cdot \sqrt{difficulté}$).
 - Baisse de la scalabilité de l'or des monstres selon le niveau du joueur

$(Or \cdot \text{niveau} \rightarrow Or \cdot \sqrt{\text{niveau}})$.

— **Version 3.0** $\rightarrow$ **Version Finale** :

— Ajout d'un multiplicateur de vie rendu d'une potion selon le niveau du joueur. $(GAIN_POTION \rightarrow GAIN_POTION \cdot \sqrt{\text{niveau}})$.